

Solutions for Homework Assignment #2

Answer to Question 1.

ALGORITHM. The algorithm is to process videos in order of decreasing second phase length.

SCHEDULE(V)

```

1   $L :=$  the set of triples in  $V$  sorted in decreasing second phase length
2  for  $i := 1$  to  $n$  do  $S[i] :=$  first component of  $i$ -th tripe in  $L$ 
3  return  $S$ 
```

CORRECTNESS. We will prove the optimality of the schedule output by our algorithm by using the exchange argument.

Let $S = i_1, i_2, \dots, i_n$ be any schedule (i.e., permutation of $1..n$). The finish time of video i_k in this schedule is $(f_{i_1} + f_{i_2} + \dots + f_{i_k}) + s_{i_k}$. The term in parentheses is the time for the first phase of all videos up to and including i_k and the last term is the time for the second phase of video i_k , which can start as soon as its first phase is completed.

Given schedules S_1 and S_2 , we define an ***inversion*** of S_1 with respect to S_2 to be two videos that appear in a different order in S_1 than in S_2 .

Let S be any schedule produced by the greedy algorithm (there may be several, because the algorithm does not specify the order of videos with the same second-phase length) and let S^* be an optimal schedule with the minimum number of inversions wrt S . We claim that S^* has no inversions with respect to S , which implies that $S = S^*$ and therefore that the schedule produced by the greedy algorithm is optimal.

Suppose, for contradiction, that S^* has some inversion with respect to S , involving videos i' and j' . Then S^* has an inversion between two videos i and j that are consecutive in S^* : if every pair of consecutive videos between i' and j' appear in the same order in S^* as in S , then i' and j' appear in the same order in S^* as in S . Without loss of generality suppose that i appears before j in S^* . Consider now the schedule $\hat{S}$ obtained from S^* by swapping i and j (so that j appears before i in $\hat{S}$). Clearly, every video other than i and j has the same completion time in S^* as in $\hat{S}$. Let $i_1, i_2, \dots, i_k$ be the videos that precede i in S^* (and therefore j in $\hat{S}$). Consider now the completion times of i and j in each of S^* and $\hat{S}$:

Completion time of i in S^* : $C_i^* = (f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_i) + s_i$

Completion time of j in S^* : $C_j^* = (f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_i + f_j) + s_j$

Completion time of j in $\hat{S}$: $\hat{C}_j = (f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_j) + s_j$

Completion time of i in $\hat{S}$: $\hat{C}_i = (f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_j + f_i) + s_i$

Since i and j form an inversion in S^* with respect to S , these two videos do not form an inversion in $\hat{S}$ with respect to S , and therefore (since in S videos appear in order of non-increasing second-phase time) $s_j \geq s_i$. So, from the above completion times we have: (a) $C_i^* \leq C_j^*$ (since $f_j \geq 0$ and $s_i \leq s_j$), (b) $\hat{C}_j \leq C_j^*$ (because $f_i \geq 0$), and (c) $\hat{C}_i \leq C_j^*$ (because the parenthesized expressions are equal but $s_i \leq s_j$). Therefore, $\max(\hat{C}_i, \hat{C}_j) \leq C_j^* = \max(C_i^*, C_j^*)$. Since all videos other than i and j have the same completion time in $\hat{S}$ as in S^* , the maximum completion time in $\hat{S}$ is at most the maximum completion time in S^* . Since S^* is optimal, so is $\hat{S}$. But $\hat{S}$ has one inversion less than S^* with respect to S , contradicting the definition of S^* as an optimal schedule with the fewest inversions with respect to S .

RUNNING TIME. The running time of the algorithm is dominated by the time to sort V , which can be done in $O(n \log n)$ time using, say, mergesort or heapsort.

Answer to Question 2.

ALGORITHM. Intuitively, the algorithm works as follows: We let ℓ be the maximum time slot in which we can schedule an order; initially this is the slot $\max\{d_i : 1 \leq i \leq n\}$. Among all remaining (i.e., not previously scheduled) orders that can be scheduled in slot ℓ (i.e., orders whose deadline is ℓ or more), we choose the most profitable one, with ties broken arbitrarily. Now the slots left in which we can still schedule the remaining orders are the slots $1.. \ell - 1$. In fact, we can (and for reasons of running time efficiency we should) eliminate from consideration slots in the range $1.. \ell - 1$ that are greater than the maximum deadline of the remaining orders. We continue scheduling the remaining orders in these slots using the same logic, until we run out of slots or orders. This is expressed in pseudocode as follows:

```

SCHEDULE( $J$ )
1  for  $i := 1$  to  $n$  do  $S(i) := \infty$                                  $\triangleright$  initialization
2   $\ell := \infty$ 
3  while  $\ell > 1$  and  $J \neq \emptyset$  do
4       $\ell := \min(\ell - 1, \max\{d_i : (i, d_i, p_i) \in J\})$            $\triangleright$  last slot to schedule orders left in  $J$ 
5       $(j, d_j, p_j) :=$  most profitable order in  $J$  with deadline  $\geq \ell$   $\triangleright$  choose order to schedule in  $\ell$ 
6       $S(j) := \ell$                                                      $\triangleright$  schedule  $j$  in slot  $\ell$ 
7       $J := J - \{(j, d_j, p_j)\}$                                         $\triangleright$  delete schedule  $j$  in slot  $\ell$ 
8  return  $S$ 

```

CORRECTNESS. Intuitively, the algorithm is correct because it schedules as late as possible the most profitable order available for scheduling, as as to leave as much room as possible to schedule the remaining orders. We will use the “promising set” method to prove that it produces an optimal schedule, where the “set” in question now is the partial schedule S that the algorithm has produced so far. So we must define what it means for a schedule S^* to “extend” another schedule S . Intuitively it means that S^* schedules every order that S schedules in the same slot as S ; it is possible that S^* schedules more orders than S . More precisely, S^* *extends* S if for every order i such that $S(i) \neq \infty$ (i.e., S schedules order i), $S^*(i) = S(i)$.

Let S_t be the schedule in variable S at the end of the t -th iteration of the loop in lines 3-7. It is obvious that S_t is indeed a schedule: it does not map distinct orders into the same slot, and it schedules every order by its deadline. We will prove:

Lemma 1. *For every iteration t of the loop, there is an optimal schedule S_t^* that extends S_t .*

Proof. By induction on the iteration number t . The basis is trivially true because $S_0(i) = \infty$ for every order i , so it is extended by any schedule.

For the induction step, suppose that there is an optimal schedule S_t^* that extends S_t . We will prove that, if iteration $t + 1$ exists, then there is optimal schedule S_{t+1}^* that extends S_{t+1} . Let j be the order that is scheduled in iteration $t + 1$, and ℓ be the slot in which order j is scheduled in that iteration. There are two exhaustive cases depending on whether j is scheduled in S_t^* (i.e., whether $S_t^*(j) = \infty$).

CASE 1. $S_t^*(j) = \infty$ (i.e., j is not scheduled in S_t^*). Then some order j' is scheduled in slot ℓ in S_t^* , i.e., $S_t^*(j') = \ell$. (Otherwise we could schedule j in slot ℓ and improve the profit, contradicting that S_t^* is optimal). Let S_{t+1}^* be the function that is identical to S_t^* except that instead of j' we schedule j in slot ℓ : $S_{t+1}^*(j) = \ell = S_{t+1}(j)$ and $S_{t+1}^*(j') = \infty$. It is easy to check that this is a schedule. Note that when the greedy algorithm added j to the schedule in iteration $t + 1$, j' was still in the set of unscheduled orders J . (Otherwise, we would have $S_t(j') = \ell'$ for some $\ell' > \ell$, contradicting that S_t^* extends S_t .) Also note that, since j' is scheduled in slot ℓ in S_t^* , $d_{j'} \geq \ell$. Since (a) $j' \in J$ during iteration $t + 1$, (b) $d_{j'} \geq \ell$, and (c) the greedy algorithm added j , rather than j' , to the schedule in iteration $t + 1$, it follows that $p_j \geq p_{j'}$. Thus, S_{t+1}^* is at least as profitable as S_t^* , and since S_t^* is optimal so is S_{t+1}^* . By its definition and the induction hypothesis that S_t^* extends S_t , S_{t+1}^* extends S_{t+1} . So, S_{t+1}^* is an optimal schedule that extends S_{t+1} , as wanted.

CASE 2. $S_t^*(j) = \ell'$ for some $\ell' \neq \infty$ (i.e., j is scheduled in some slot ℓ' in S_t^*). If $\ell' = \ell$ then we take $S_{t+1}^* = S_t^*$, and we are done: We have an optimal schedule that extends S_{t+1} .

So, in the rest of the proof assume $\ell' \neq \ell$. There are two exhaustive subcases, depending on whether in S_t^* some order is scheduled in slot ℓ

- SUBCASE 2A. No order is scheduled in slot ℓ in S_t^* (i.e., there is no j' such that $S_t^*(j') = \ell$). Then let S_{t+1}^* be identical to S_t^* , except that S_{t+1}^* schedules j in slot ℓ instead of slot ℓ' ; i.e., $S_{t+1}^*(j) = \ell = S_{t+1}(j)$. It is clear that S_{t+1}^* is a schedule and has the same profit as S_t^* , so it is optimal. Furthermore, it extends S_{t+1} (by its definition and the induction hypothesis that S_t^* extends S_t). Thus, S_{t+1}^* is an optimal schedule that extends S_{t+1} , as wanted.
- SUBCASE 2B. Some order j' is scheduled in slot ℓ in S_t^* (i.e., for some j' , $S_t^*(j') = \ell$). We will prove that

$$\ell > \ell' \quad (*)$$

Suppose, for contradiction, that $\ell' > \ell$ (recall that $\ell' \neq \ell$). Since the algorithm schedules order j in iteration $t+1$, $j \in J$ before that iteration. And since j is scheduled in slot ℓ' in S_t^* , $d_j \geq \ell'$. Therefore, the algorithm considers slot ℓ' before iteration $t+1$ (i.e., it does not skip it in line 4). When the algorithm considered slot ℓ' , j was still in J at that time, and $d_j \geq \ell'$ (as we just argued); so, the algorithm schedules some order j'' in slot ℓ' . Since S_t^* extends S_t , S_t^* also schedules j'' in slot ℓ' . So, by the hypothesis of Case 2, $j'' = j$. Thus, the algorithm schedules order j in slot ℓ' . But this contradicts that the algorithm schedules j in slot $\ell \neq \ell'$. This proves (*).

So S_t^* schedules order j' in slot ℓ and order j in slot $\ell' < \ell$. Now let S_{t+1}^* be identical to S_t^* except that the slots of j and j' are swapped: $S_{t+1}^*(j) = \ell$ and $S_{t+1}^*(j') = \ell'$. S_{t+1}^* is a schedule because (a) $d_j \geq \ell$ (since S_{t+1} schedules j in slot ℓ), and (b) $d_{j'} \geq \ell'$ (since S_t^* schedules j' in $\ell > \ell'$). S_{t+1}^* has the same profit as S_t^* and so it is also optimal. Finally, by its definition and the induction hypothesis that S_t^* extends S_t , S_{t+1}^* extends S_{t+1} . So, S_{t+1}^* is an optimal schedule that extends S_{t+1} , as wanted. $\square$

First we note that the loop terminates because in each iteration an order is removed from J , so the loop cannot be executed more than n times. Let S be the schedule returned by the algorithm. By the lemma, there is an optimal schedule S^* that extends S . We claim that $S^* = S$. Suppose, for contradiction, that $S^* \neq S$. Then (since S^* extends S) there is some order j such that $S(j) = \infty$ and $S^*(j) = \ell \neq \infty$. Since $S(j) = \infty$, j is in J at the end of the algorithm and therefore it is in J in every iteration. Since $S^*(j) = \ell$, $d_j \geq \ell$ and so the algorithm considered slot ℓ (it did not skip it), yet it did not schedule any order in that slot. Since j is always in J , the only reason why no order was scheduled in slot ℓ by the algorithm is that $d_j < \ell$. But this contradicts that S^* schedules order j in slot ℓ . Thus, $S^* = S$ and the algorithm returns an optimal schedule.

RUNNING TIME. Line 1 takes $O(n)$ time and line 2 takes $O(1)$ time. As noted above, the loop in line 3 is executed at most n times. Lines 4 and 5 can be done in $O(n)$ time each (by scanning the set of remaining orders J). Lines 6 and 7 can be done in $O(1)$ time each. So, each iteration of the loop takes $O(n)$ time and the entire algorithm runs in $O(n^2)$ time.

The algorithm can be implemented in $O(n \log n)$ time by considering the orders in decreasing deadline, starting with the maximum time slot that can be used to schedule an order, and keeping all the orders with deadline greater than or equal to the maximum slot available to schedule the remaining orders in a max-heap keyed on profit. The details are interesting and are left as an exercise.

Answer to Question 3.

We use Dijkstra's algorithm to find a minimum weight path from s to each node and, by reversing the edges, a minimum weight path from each node to t . Then we consider each potential edge (u, v) to determine a shortest $s \rightarrow t$ path that uses that potential edge (by computing the sum of the minimum weight $s \rightarrow u$ path, the minimum $v \rightarrow t$ path, and the weight of the edge (u, v)) and pick the potential edge that minimizes this. This is described in the pseudocode below:

```
BESTPOTENTIALEDGE( $G, s, t, E', \mathbf{wt}$ )
1   $d := \text{DIJKSTRA}(G, s, \mathbf{wt})$ 
2   $G^R := G$  with the edges reversed
3  for each  $(u, v) \in E$  do  $\mathbf{wt}^R(v, u) := \mathbf{wt}(u, v)$ 
4   $d^R = \text{DIJKSTRA}(G^R, t, \mathbf{wt}^R)$ 
5   $best := \text{NIL}; \min\_wt := \infty$ 
6  for each  $(u, v) \in E'$  do
7      if  $d(u) + d^R(v) + \mathbf{wt}(u, v) < \min\_wt$  then
8           $best := (u, v); \min\_wt := d(u) + d^R(v) + \mathbf{wt}(u, v)$ 
9  return  $best$ 
```

Each use of Dijkstra's algorithm in lines 1 and 4 takes $O((m+n) \log n)$ time. Reversing the edges and setting the weights of the reversed edges in lines 2 and 3 takes $O(m+n)$ time (this is an easy CSCB63 exercise). The loop in line 6 takes $O(k)$ time, and line 9 takes $O(1)$ time. So, the overall running time of the algorithm is $O(k + (m+n) \log n)$ time.